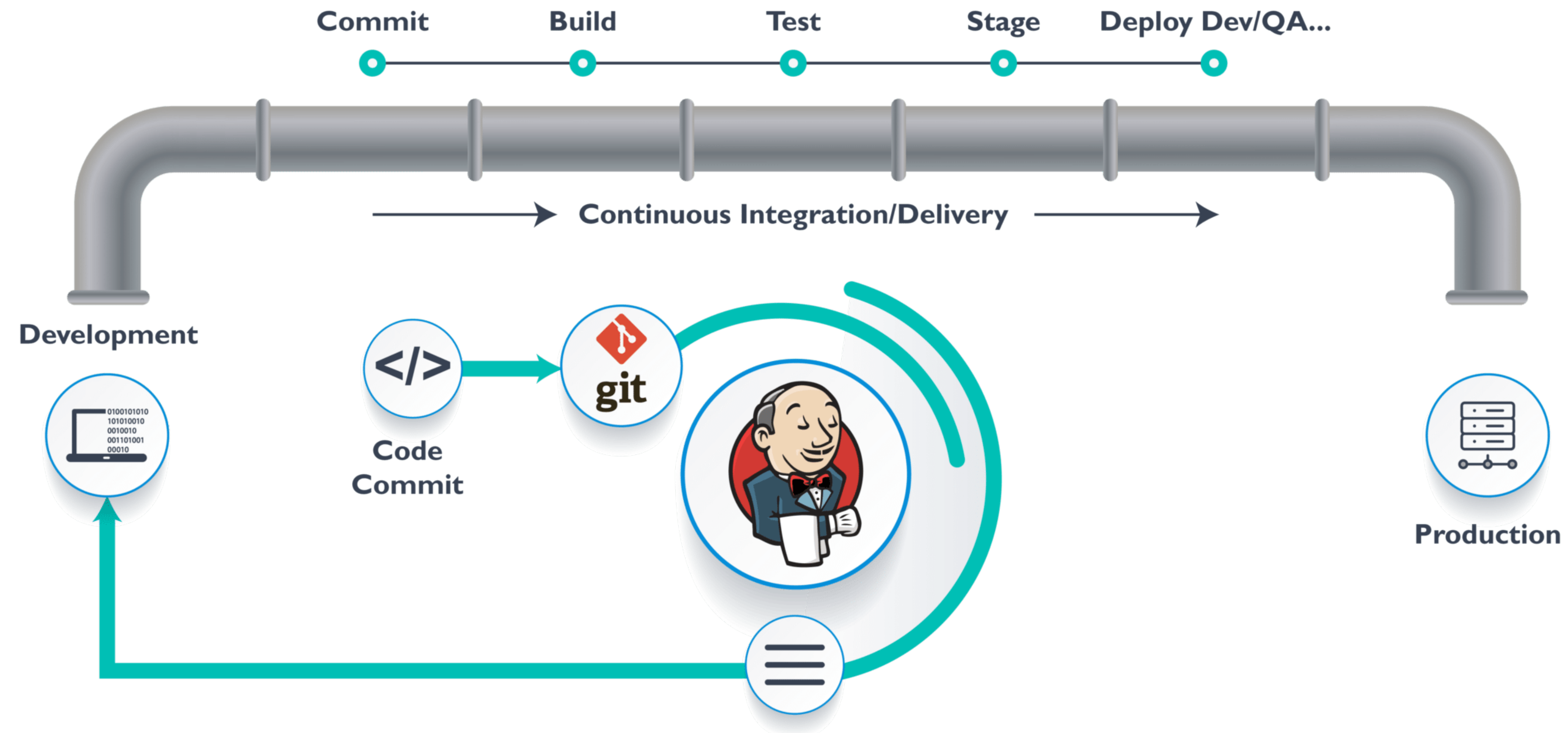


Software Dependability

Test Case Assessment and Test-Driven Development

Test Case Assessment

Developers should depend on an effective test suite that can be used to exercise their code after **every commit**.



Test Code Quality and Effectiveness: Writing Effective Tests

The secret to writing effective tests is to **automate them**.

Automation refers to the use of software to control the test **execution**, the **comparison** of actual outcomes to expected outcomes, the **setting up** of test preconditions, and other test **control** and test **reporting** functions.

The benefits of test automation are **manifold**:

1. The overall cost of software verification and validation is reduced.
2. The error-proneness of humans is reduced.
3. The cost of doing regression testing is reduced.

Software testability indicates the degree to which a system or component facilitates the **establishment of test criteria** and the **performance of tests** to determine whether those criteria have been met.

Software testability is dominated by two critical problems: **controllability** and **observability**:

- **Controllability.** How easy it is to **provide** a program with the **needed inputs** regarding values, operations, and behaviors.
- **Observability.** How easy it is to **observe the behavior** of a program in terms of its outputs, effects on the environment, and other hardware and software components

Input from hardware sensors or distributed software is harder to provide

Software that affects hardware devices, databases, or remote files have low observability

Test Code Quality and Effectiveness: How to Measure Them

Once generated test cases are automated, the next question is how to assess their **ability to find defects**.

Dijkstra's law. Because we cannot do testing exhaustively, we can only sample test cases. Therefore, we can never be sure that our program is defect-free. Because showing the absence of defects is impossible, testing aims to demonstrate their presence. **Testing is successful if we find defects.**

Hence, **we can only estimate** the ability of tests to find defects.
The most widely known way is to compute **test code coverage**.

A **coverage criterion** describes a finite subset of test cases out of the vast/infinite number of possible tests.

A coverage criterion can be viewed as a **finite set of test requirements that a test suite must satisfy**. There is usually more than one way to cover a test requirement, so a coverage criterion is not a unique test suite description.

Coverage criteria serve two primary purposes:

- **Test adequacy:** Have I got enough tests?
- **Guidance:** Where should I test more?

Test Code Quality and Effectiveness: Measuring Code Coverage

To use coverage in practice, we need to measure it.

We can **instrument the source code** with a probe for every test requirement described by the coverage criterion.

When test cases are run on the instrumented program, the instrumentation keeps track of what has been executed. At the end of execution, we can analyze this information **to identify uncovered areas and quantify coverage**.

```
public int min(A, B) {  
    int m = A;  
    if(A > B) {  
        m = B;  
    }  
    return m;  
}
```



```
public int min(A, B) {  
    int m = A;  
    if(A > B) {  
        Mark: "if body reached"  
        m = B;  
    }  
    return m;  
}
```

Instrumentation adds program code that **does not change functional behavior** but collects information.

However, it **can change other aspects of the program**, such as timing and interleaving.

Test Code Quality and Effectiveness: Measuring Code Coverage

To use coverage in practice, we need to measure it.

We can **instrument the source code** with a probe for every test requirement described by the coverage criterion.

When test cases are run on the instrumented program, the instrumentation keeps track of what has been executed. At the end of execution, we can analyze this information **to identify uncovered areas and quantify coverage**.

```
public int min(A, B) {  
    int m = A;  
    if (A > B) {  
        m = B;  
    }  
    return m;  
}
```

```
public int min(A, B) {  
    statements[0]++;  
    int m = A;  
    statements[1]++;  
    if (A > B) {  
        statements[2]++;  
        m = B;  
    }  
    statements[3]++;  
    return m;  
}
```

Before executing a statement, we verify that it has been executed, for example, by adding a static method call or incrementing a counter.

Test Code Quality and Effectiveness: Measuring Code Coverage

To use coverage in practice, we need to measure it.

We can **instrument the source code** with a probe for every test requirement described by the coverage criterion.

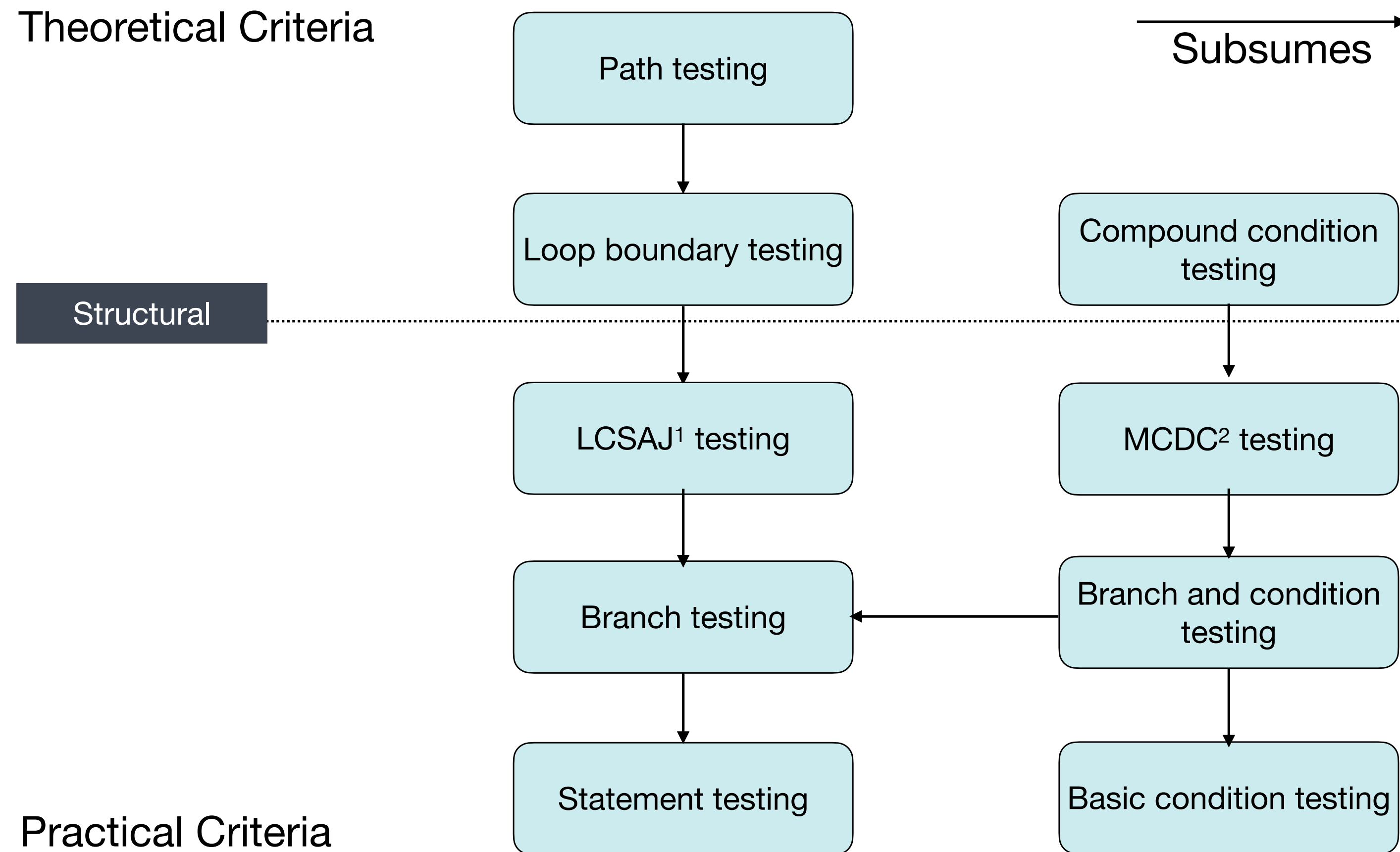
When test cases are run on the instrumented program, the instrumentation keeps track of what has been executed. At the end of execution, we can analyze this information **to identify uncovered areas and quantify coverage**.

$$coverage = \frac{\# \text{ covered test requirements}}{\# \text{ total test requirements}}$$

Usually, coverage is quantified as the percentage of test requirements covered. Nevertheless, what does it mean?

Weyuker's Hypothesis. The adequacy of a coverage criterion can only be intuitively defined.

Test Code Quality and Effectiveness: Measuring Code Coverage



¹Linear code sequence and jump

2Modified condition/decision coverage

Test Code Quality and Effectiveness: Enhancing the Assessment

The primary goal of test cases is to **find defects**.

We cannot know whether and what defects will arise in production code at design time.

However, **we know the defects we have made in the past!**

Key idea: Learning from earlier mistakes to prevent them from happening again.

Key technique: Simulate earlier mistakes and see whether the available test suite finds the resulting defects.

This concept is the basis for *fault-based* testing, also known as **mutation testing**.

Suppose we have a program under test and test cases that exercise it. The program passes all tests.

Mutation testing aims to insert simple faults in the program, creating the so-called **mutants** to assess test quality.

```
int doSomething(int x, int y) {  
    if(x < y)  
        return x+y;  
    else  
        return x*y;  
}
```

Production code

```
int a = doSomething(5, 10);  
assert(a, 15);
```

Test code



```
int doSomething(int x, int y) {  
    if(x < y)  
        return x-y;  
    else  
        return x*y;  
}
```

Mutant

```
int a = doSomething(5, 10);  
assert(a, 15);
```

Test code



If the test fails, the test case is robust enough to detect the fault. In this case, we say that the test case killed the mutant.

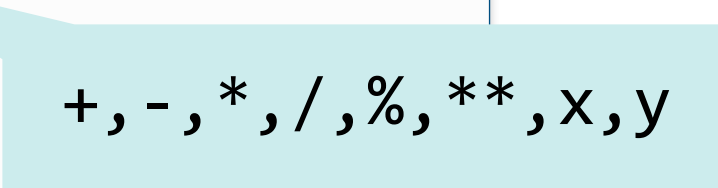
Otherwise, we need to improve the test suite.

Test Code Quality and Effectiveness: Mutation Testing

- **Mutant.** A slightly artificially changed version of the production code.
- **Syntactic change.** A valid, i.e., compilable, change to production code.
- **Mutation operator.** A rule to derive mutants from a program.

Some Mutation Operators

```
int doSomething(int x, int y) {  
    if(x < y)  
        return x+y;  
    else  
        return x*y;  
}
```



+, -, *, /, %, **, x, y

Arithmetic Operator Replacement. AOR not only replaces the operators but also the two exceptional cases where $x + y$ is mutated to x and y , i.e., dropping the other operand.

```
int doSomething(int x, int y) {  
    if(x < y)  
        return x+y;  
    else  
        return x*y;  
}
```



abs(variable)

Absolute Value Insertion. An absolute value is added in an arbitrary code location, including in return statements.

Test Code Quality and Effectiveness: Mutation Testing

- **Mutant.** A slightly artificially changed version of the production code.
- **Syntactic change.** A valid, i.e., compilable, change to production code.
- **Mutation operator.** A rule to derive mutants from a program.

Some Mutation Operators

```
int doSomething(int x, int y) {  
    if(x < y)  
        return true;  
    else  
        return x*y;  
}
```

<, >, <=, >=, =, !=, false, true

Relational Operator Replacement. ROR replaces operators with other operators, including cases where the full relational expression is replaced with true or false.

```
int doSomething(int x, int y) {  
    if(x & y)  
        // some code..  
    return x;  
}
```

&&, ||, &, |, ^, false, true, x, y

Conditional Operator Replacement. An operator is replaced with other operators, including cases of true/false or single variables.

Test Code Quality and Effectiveness: Mutation Testing

OO Mutation Operators

- HVD - Hiding Variable Deletion
- TKD - this Keyword Deletion
- AMC - Access Modifier Change
- HVI - Hiding Variable Insertion
- OMD - Overriding Method Deletion
- OMM - Overridden Method Moving
- OMR - Overridden Method Rename
- SKR - Super Keyword Deletion
- PCD - Parent Constructor Deletion
- ATC - Actual Type Change
- DTC - Declared Type Change
- PTC - Parameter Type Change
- RTC - Reference Type Change
- OMC - Overloading Method Change
- OMD - Overloading Method Deletion
- AOC - Argument Order Change
- ANC - Argument Number Change
- TKD - this Keyword Deletion
- SMV - Static Modifier Change
- VID - Variable Initialization Deletion
- DCD - Default Constructor 2

Interface Mutation

It covers integration testing:

- Change a calling method by modifying the parameters.
- Change a calling method by invoking another method.
- Change a called method by modifying its input/output parameters.

First vs Higher Order

First-order mutants are programs where only one mutation is applied.
Higher-order are mutants of mutants.

Mutation coverage

$$mutation\ coverage = \frac{\# \text{ killed mutants}}{\# \text{ total mutants}}$$

Test Code Quality and Effectiveness: Problems of Mutation Testing

The main problem of mutation testing is **scalability**.

Modern systems are large and have thousands of tests: mutation analysis requires (i) the **generation** of mutants; (ii) the **compilation** of mutants; (iii) the **execution** of test cases against the mutants. Hence, it is not practical!

Equivalent Mutants. A mutation is a **syntactic change**. However, it may leave **semantics unchanged**!

```
int max(int[] values) {  
    int r, i;  
    r = 0;  
    for(i=1; i<values.length; i++) {  
        if(values[i] > values[r])  
            r = i;  
    }  
  
    return values[r];  
}
```

It merely presents an extra comparison of the first element to itself, which cannot alter the functional behavior.

```
int max(int[] values) {  
    int r, i;  
    r = 0;  
    for(i=0; i<values.length; i++) {  
        if(values[i] > values[r])  
            r = i;  
    }  
  
    return values[r];  
}
```

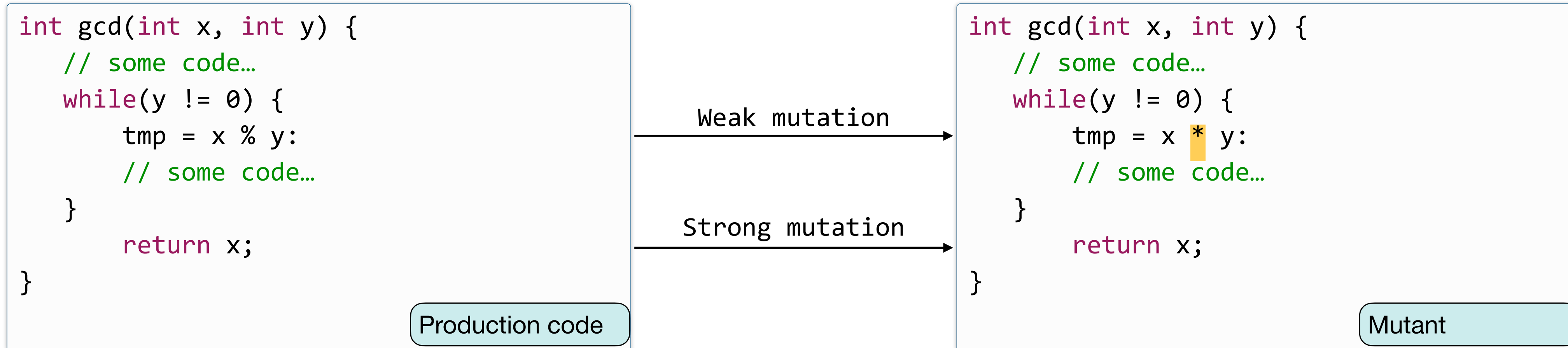
Test Code Quality and Effectiveness: Problems of Mutation Testing

The main problem of mutation testing is **scalability**.

Modern systems are large and have thousands of tests: mutation analysis requires (i) the **generation** of mutants; (ii) the **compilation** of mutants; (iii) the **execution** of test cases against the mutants. Hence, it is not practical!

Equivalent mutants provide no insight, yet they increase the cost of mutation analysis. Unfortunately, **finding equivalent mutants is an undecidable problem**.

Weak versus Strong Mutation. Strong mutations are operators that propagate the artificial fault to some observable behavior. Weak mutations are located within a specific environment.



Test Code Quality and Effectiveness: Problems of Mutation Testing

The main problem of mutation testing is **scalability**.

Modern systems are large and have thousands of tests: mutation analysis requires (i) the **generation** of mutants; (ii) the **compilation** of mutants; (iii) the **execution** of test cases against the mutants. Hence, it is not practical!

Equivalent mutants provide no insight, yet they increase the cost of mutation analysis. Unfortunately, **finding equivalent mutants is an undecidable problem**.

Weak versus Strong Mutation. Strong mutations are operators that propagate the artificial fault to some observable behavior. Weak mutations are located within a specific environment.

The selection and execution of mutation operators **affect** the performance of mutation analysis and testing.

It is possible to reduce computation costs by applying

- “*fewer*” techniques to select a subset of mutants to evaluate;
- “*smarter*” techniques that exploit runtime information to avoid unnecessary test executions;
- “*faster*” techniques to reduce the execution time of single mutants.

More recently, **machine learning** has been used to statistically predict the **mutation score** of test cases.

Test Code Quality and Effectiveness: Complementing the Assessment

The primary goal of testing is to **find actual defects**.

Unfortunately, tests could give false positives because **they are defective!**

Besides errors developers make when implementing test suites, a common issue affecting tests is “**flakiness**”.

Test flakiness. Test cases are flaky if they sometimes pass and sometimes fail with the same code.

```
public void testRsReportsWrongServerName() throws Exception {
    MiniHBaseCluster cluster = TEST_UTIL.getHBaseCluster();
    MiniHBaseClusterRegionServer firstServer = (MiniHBaseClusterRegionServer) cluster.getRegionServer(0);

    HServerInfo hsi = firstServer.getServerInfo();
    firstServer.setHServerInfo(...);

    // Sleep while the region server pings back
    Thread.sleep(2000);
    assertTrue(firstServer.isOnline());
    assertEquals(2, cluster.getLiveRegionServerThre
    // similarly for secondServer
}
```

If the server does not ping back in time, the test fails; otherwise, it passes.

Test Code Quality and Effectiveness: Relevance of Test Flakiness

Flaky Tests at Google

Google has around 4.2 million tests that run on our continuous integration system. Of these, around 63 thousand have a flaky run over the course of a week. While this represents less than 2% of our tests, it still causes significant drag on our engineers.

<https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html>

John Micco, Former Test Manager at Google - currently at Netflix

“We are interested in proposals that tackle any topics on Testing and Verification [...] and in particular, we welcome proposals that tackle [...] **test flakiness**.”

<https://research.fb.com/blog/2019/05/facebook-launches-testing-and-verification-research-awards-at-icse-2019/>

Facebook launches testing and verification research awards at ICSE 2019



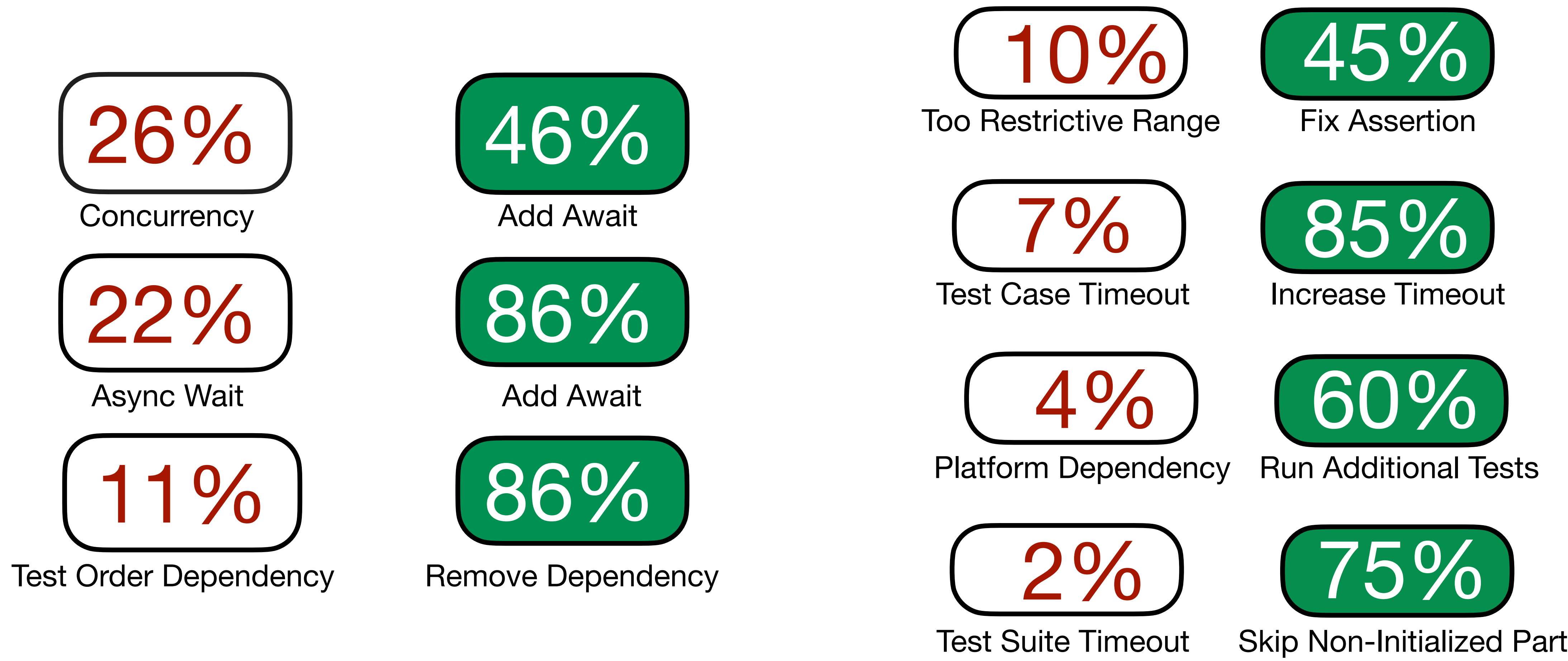
Test Flakiness – Methods for identifying and dealing with flaky tests

Posted on November 18, 2019 by [Jason Palmer](#)

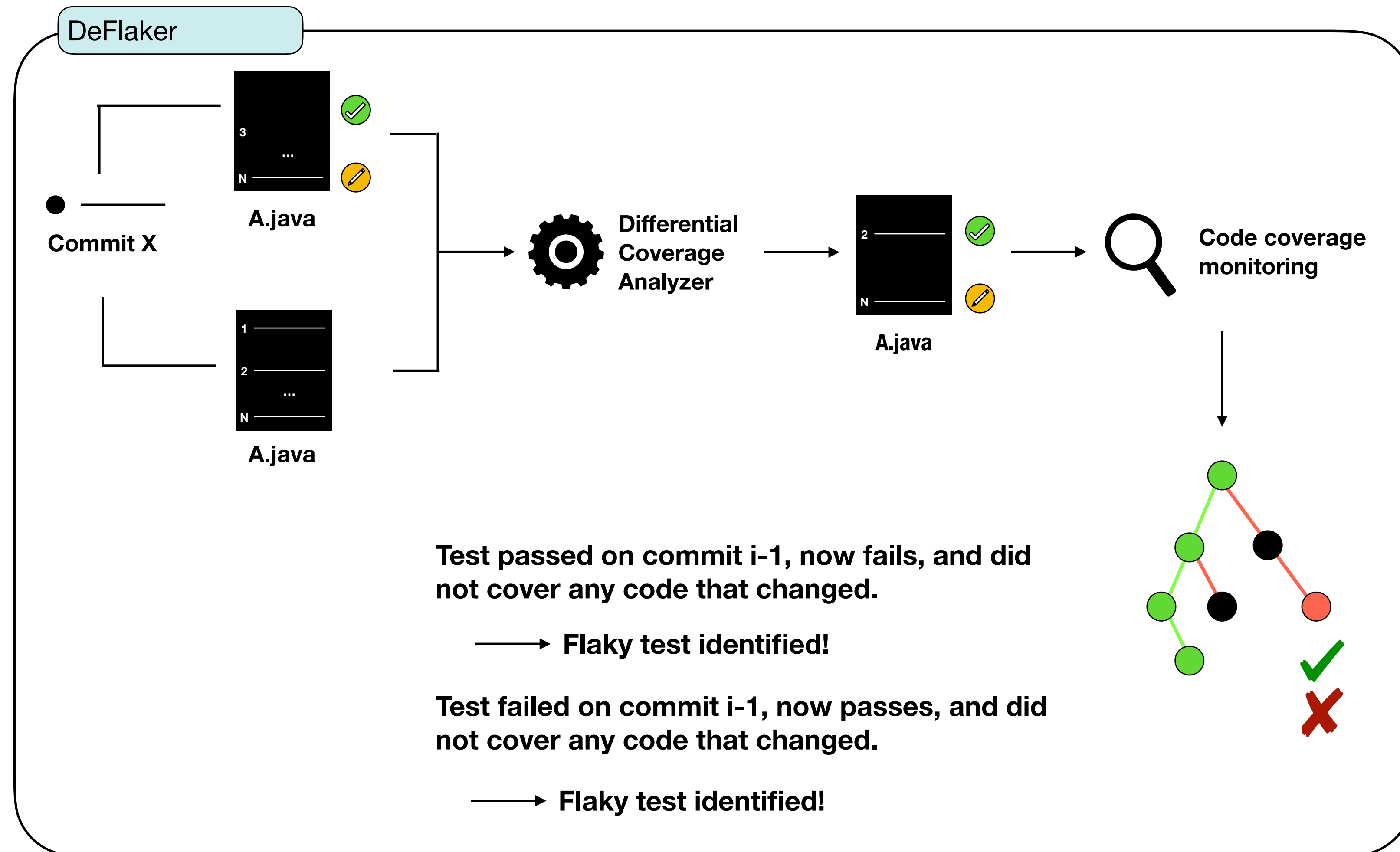
<https://labs.spotify.com/2019/11/18/test-flakiness-methods-for-identifying-and-dealing-with-flaky-tests/>

Jason Palmer, PM of Web Infrastructure and Toast Infrastructure at Spotify

Test Code Quality and Effectiveness: Causes of Test Flakiness



Test Code Quality and Effectiveness: Detecting Test Flakiness



Bell, J., Legunsen, O., Hilton, M., Eloussi, L., Yung, T., & Marinov, D. (2018, May). DeFlaker: Automatically detecting flaky tests. In Proceedings of the 40th international conference on software engineering (pp. 433-444).

Software Testing: the Development Process (from the Testing Perspective)

So, when should one start testing? It depends.

- An **early start of testing reduces cost and time** to produce error-free software.
- The **development lifecycle** of the project can influence the decision.
- The **available resources** can influence the decision.
- Additional **project-specific needs** can influence the decision.



Developers can test **while** programming new features.

Developers can program new software **driven** by testing requirements to make the software work immediately.

Test cases can serve as **documentation** to understand better how the production code would look.

Software Testing: Test-Driven Development

If we can use tests as documentation, why **not be driven by them** while producing new production code?

Test-Driven Development (TDD) is “*a technique for building software **that guides software development by writing tests***” - Martin Fowler.

Test-Driven Development is NOT primarily about testing or development, but rather about **design** and how to **evolve it through refactoring**.

From a practical point of view, **developers (NOT testers) write unit tests and then code**.

In this way, development and testing become two sides of the same coin.

Writing tests after

Testing does not give direct feedback to design and programming.

Most of the time, after implementing some code, unit testing is omitted.

Writing tests after developing code often results in “happy path” testing, i.e., biased test design.

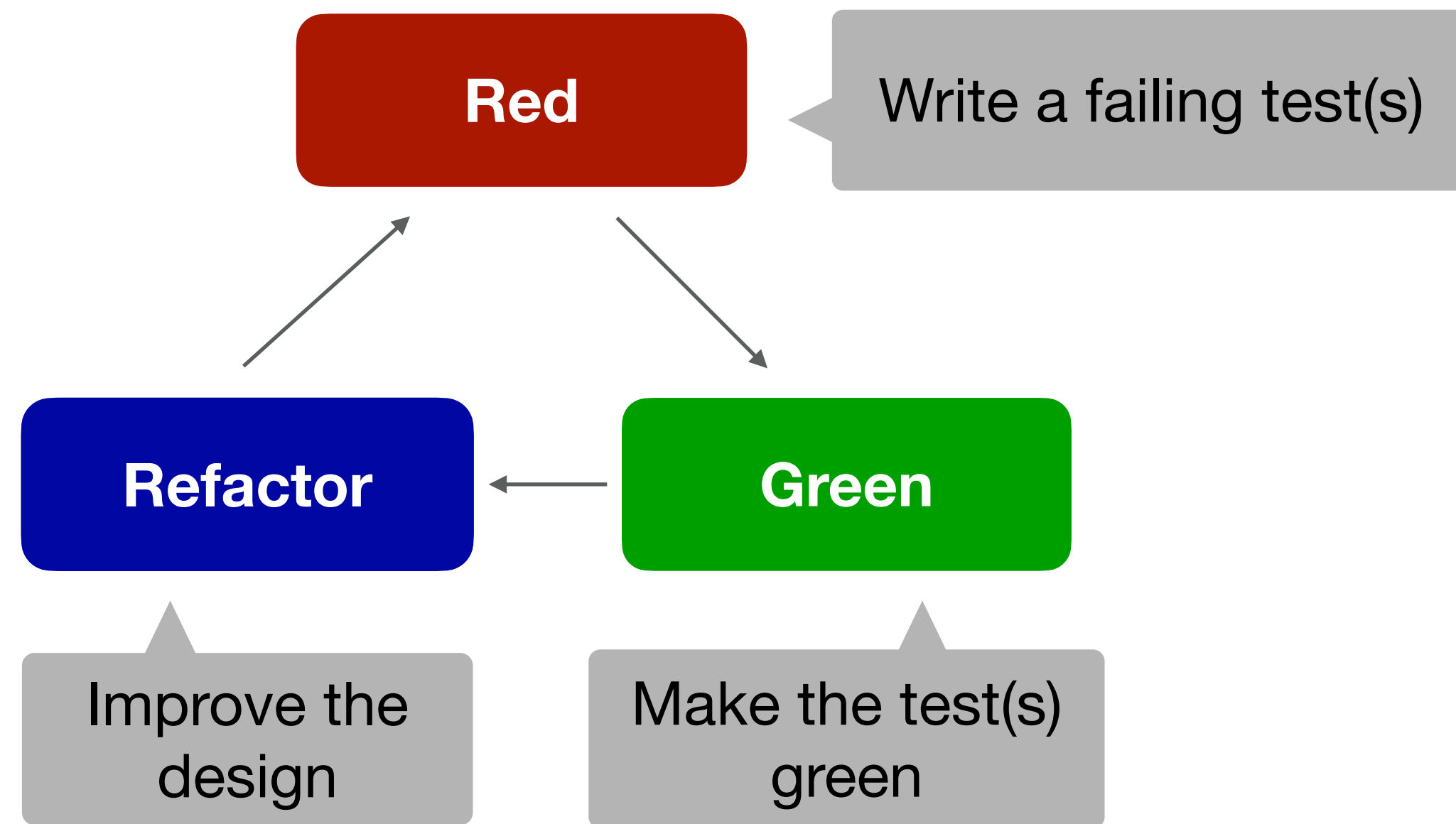
Writing tests first

The feedback is directly fed back into improving design and programs.

TDD inverts this sequence and helps create unit tests immediately.

TDD allows developers to reason about the ideal behavior of production code.

Test-Driven Development: The Basic Cycle



1. Write a little test that does not work or even compile at first.
2. Make the test work quickly, committing whatever sins are necessary.
3. Eliminate all the duplication and smells created in just getting the test to work.

Best practices

Make it green, then make it clean!

It is similar to a game where you aim to play with the test until it becomes green.

The Three Laws.

Law 1: You may **not write production code unless you have first written a failing unit test.**

Law 2: You may **not write more of a unit test than is sufficient to fail.**

Law 3: You may **not write more production code than is sufficient to make the failing test pass.**

Test-Driven Development: Fundamental Principles

- Follow the **TDD cycle** and the **three laws**.
- Never write a new functionality **without** a failing test.
- Continuously make small, **incremental changes**.
- **Keep the system running at all times.**
No one can make changes that break the system, and failures must be addressed immediately.
- **TDD is a collaborative process**, especially at the beginning.
It is good to have regular meetings among developers, testers, and business analysts.

Test-Driven Development: Benefits for Design and Implementation

When writing tests.

[Design] Makes developers clarify the acceptance criteria for the next piece of work.

[Design] Encourages developers to write loosely coupled components so they can be easily tested in isolation and, at higher levels, combined.

[Design] Adds an executable description of the behavior of the source code.

[Implementation] Adds a complete regression test suite.

When executing tests.

[Design] Let developers know when they have done enough to discourage unnecessary features.

[Implementation] Detects errors while the context is fresh in the developers' minds.

Consequences.

A recent study has quantified that 45% to 90% of potential defects can be avoided through test-driven development.

Developers are up to 10 times faster in finding defects in production code.

Test-Driven Development: an Example

```
// LeapYear.spec.js
describe('Leap year calculator', () => {
  it('should consider 1996 as leap', () => {
    expect(LeapYear.isLeap(1996)).toBe(true);
  });
});
```

Red



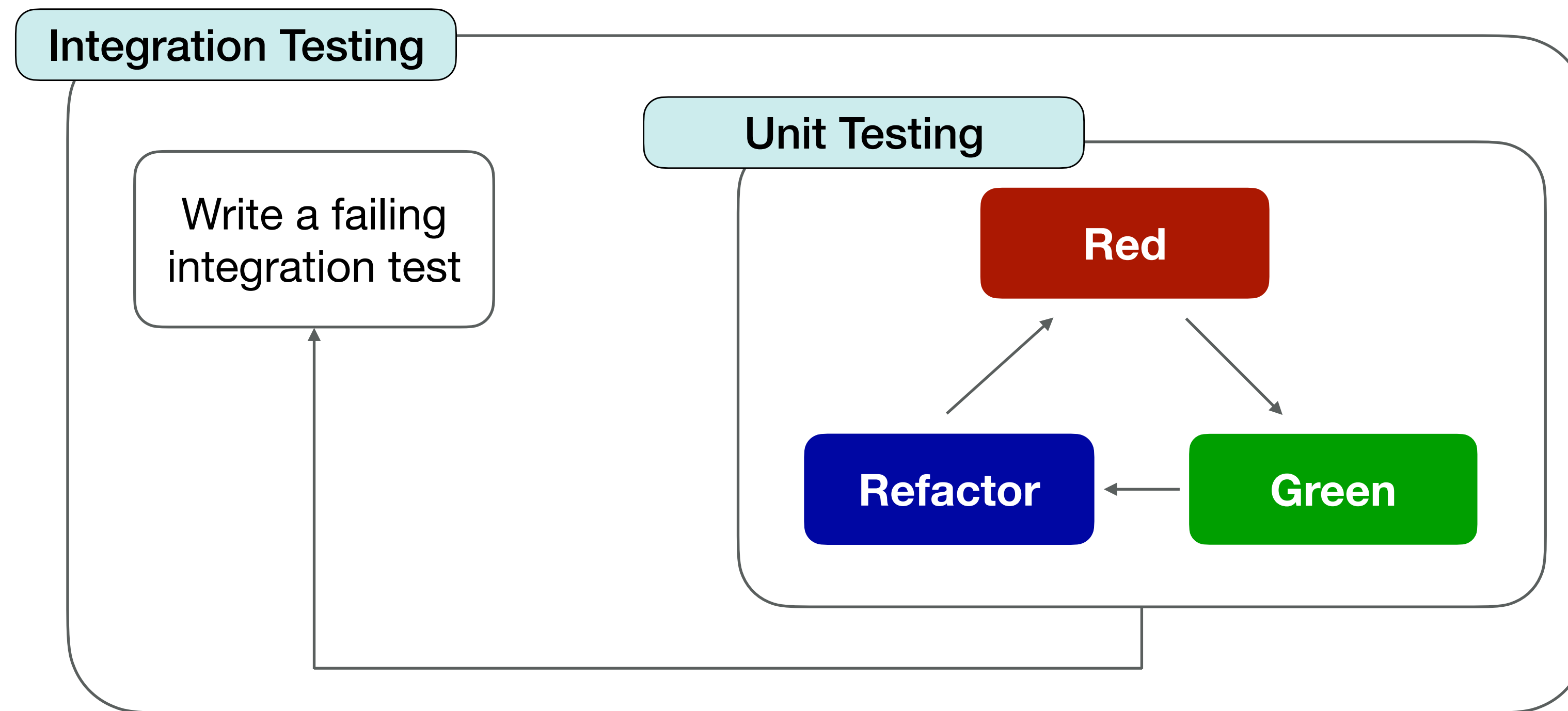
<https://jasmine.github.io>

What do you learn from this test? What will the corresponding production code look like?
Can the test be used as documentation for how to write production code?

- The function **isLeap(...)** should return **true** for 1996, i.e., 1996 is a leap year.
- The name of the leap year calculator is **LeapYear**.
- **isLeap(...)** is a static method of **LeapYear**.
- **isLeap(...)** takes a number as an argument and returns **true** or **false**.

Test-Driven Development: the Extended Cycle

Test-driven development can be applied at all testing levels (e.g., integration).



NB: The higher the level, the higher the difficulties in applying a TDD approach. Developers with less experience with TDD might require some time.

Test-Driven Development: the F.I.R.S.T. Approach

According to the test-driven development principles, each test case should be developed aiming at five characteristics:

- **F**ast: it should exercise the production code as quickly as possible.
- **I**ndependent: It should not have dependencies on other tests.
- **R**epeatable: It should be repeated every time the production code is modified and should be deterministic (not flaky).
- **S**elf-validating: No manual inspection is required to check whether the test has passed or failed.
- **T**horough: It should cover every use case scenario and NOT just aim for 100% coverage.

Test-Driven Development: the F.I.R.S.T. Approach

According to the test-driven development principles, each test case should be developed aiming at five characteristics:

- **Fast:** it should exercise the production code as quickly as possible.
- **Independent:** It should not have dependencies on other tests.
- **Repeatable:** It should be repeated every time the production code is modified and should be deterministic (not flaky).
- **Self-validating:** No manual inspection is required to check whether the test has passed or failed.
- **Thorough:** It should cover every use case scenario and NOT just aim for 100% coverage.

Why?

You need immediate feedback.

Ensure tests can be added to your continuous integration pipeline.

So what?

Design your test to be “simple”, avoid expensive computations by, for instance, mocking dependencies.

Test-Driven Development: the F.I.R.S.T. Approach

According to the test-driven development principles, each test case should be developed aiming at five characteristics:

- **F**ast: it should exercise the production code as quickly as possible.
- **I**ndependent: It should not have dependencies on other tests.
- **R**epeatable: It should be repeated every time the production code is modified and should be deterministic (not flaky).
- **S**elf-validating: No manual inspection is required to check whether the test has passed or failed.
- **T**horough: It should cover every use case scenario and NOT just aim for 100% coverage.

Why?

You should run tests in isolation to check the status of a specific piece of code.

You should avoid tests that depend on the results of other tests.

So what?

Design your test to be self-contained and avoid the use of external resources.

Test-Driven Development: the F.I.R.S.T. Approach

According to the test-driven development principles, each test case should be developed aiming at five characteristics:

- **Fast:** it should exercise the production code as quickly as possible.
- **Independent:** It should not have dependencies on other tests.
- **Repeatable:** It should be repeated every time the production code is modified and should be deterministic (not flaky).
- **Self-validating:** No manual inspection is required to check whether the test has passed or failed.
- **Thorough:** It should cover every use case scenario and NOT just aim for 100% coverage.

Why?

You avoid test failures that occur intermittently with the same code.

It is a matter of trust in your test suite.

So what?

Avoid common errors, such as making asynchronous calls without waiting for a response.

Test-Driven Development: the F.I.R.S.T. Approach

According to the test-driven development principles, each test case should be developed aiming at five characteristics:

- **F**ast: it should exercise the production code as quickly as possible.
- **I**ndependent: It should not have dependencies on other tests.
- **R**epeatable: It should be repeated every time the production code is modified and should be deterministic (not flaky).
- **S**elf-validating: No manual inspection is required to check whether the test has passed or failed.
- **T**horough: It should cover every use case scenario and NOT just aim for 100% coverage.

Why?

Tests should be completely automated.

You should obtain feedback on code issues without additional work.

So what?

Automate your tests, use assertions correctly, and verify their validity.

Test-Driven Development: the F.I.R.S.T. Approach

According to the test-driven development principles, each test case should be developed aiming at five characteristics:

- **F**ast: it should exercise the production code as quickly as possible.
- **I**ndependent: It should not have dependencies on other tests.
- **R**epeatable: It should be repeated every time the production code is modified and should be deterministic (not flaky).
- **S**elf-validating: No manual inspection is required to check whether the test has passed or failed.
- **T**horough: It should cover every use case scenario and NOT just aim for 100% coverage.

Why?

Coverage does not imply defect identification.

At the end of the day, a good test identifies errors.

So what?

Consider coverage as an indicator, not a goal for blindly covering the production code.

Make tests meaningful — i.e., test the specific behavior of the code.

Test-Driven Development: Best Practices

Test naming and structure

The name of the test should describe the feature that is exercised.

The name should clearly describe the test purpose — remember: it should serve as documentation.

Each test should exercise a single aspect of the production code, e.g., it should not test two methods.

Strive for one assertion per test so you know immediately what is going wrong if the test fails.

Consider what the object does, not what the object is, i.e., think of the actual behavior of the object.

Test specification

Even seemingly trivial tests are essential; you do not know what a user can do!

Treat your tests like your production code; do not consider production code more critical!

Ensure test readability and expressiveness to improve maintainability.

Have a consistent test structure so that the test organization is coherent with the production code.

Test-Driven Development: Be Careful of Technical Debt

```
public void testPath() throws Throwable
{
    JSTerm jsTerm = new JSTerm();
    jsTerm.makeVariable();
    jsTerm.add( (Object) "");

    assertEquals(false, jsTerm.isGrounded());
    assertEquals(true, jsTerm.isVariable());
}
```

An **Eager Test** indicates a test case that exercises more than one method of the production code, hence **it is not focused on a target** and is possibly **harder to understand** in case of failure.

Tests like this require an Extract Method refactoring: each test should have only one target.

A close-up photograph of a computer keyboard. The central focus is a large, white, rectangular key with rounded corners. On this key, there is a dark blue icon of a coffee cup with three wavy lines above it representing steam. Below the icon, the word "Break" is printed in a dark blue, serif font. The key is set against a light-colored, textured keyboard surface. Other keys are visible in the background, including one with a double quote symbol and another with a four-way arrow symbol, but they are out of focus.

Break